UNIE Universidad

Facultad de Ciencias y Tecnología

Grado en Ingeniería Informática

Plantilla LaTeX

Autor: Miguel Ángel Martín Mesa

Tutor: Dr. Nombre del Tutor

Asignatura: Personal

Curso académico: 2025-2026

2026-03-15

Version: 4.0Build: 2026-03-15 03:19:31+01:00Mode: DRAFT

Dedicatoria

Dedico este trabajo a...

Agradecimientos

Quiero agradecer a...

Resumen

Aquí va el resumen del trabajo.

Palabras clave: palabra1; palabra2; palabra3

Abstract

This document presents...

Keywords: keyword1; keyword2; keyword3

Índice general

Agradecimientos **i**

Resumen **ii**

Abstract **iii**

1. Introducción **1**
1.1. Objetivo de la plantilla 1
1.2. Filosofía de diseño 1
1.3. Organización del proyecto 2
1.4. Uso recomendado 2
1.5. Metodología de documentación 2

2. Arquitectura del proyecto **4**
2.1. Principios de arquitectura 4
2.2. Estructura del proyecto 4
2.3. Archivo principal 5
2.4. Flujo de compilación 5
2.5. Responsabilidad de los módulos 5
2.6. Beneficios de la arquitectura 6

3. Cómo escribir contenido **7**
3.1. Organización de los capítulos 7
3.2. Secciones 7
3.3. Subsecciones 7
3.4. Ejemplo de estructura 8
3.5. Flujo de escritura recomendado 8
3.6. Buenas prácticas de escritura 8
3.7. Ejemplo 9

4. Floats de alto nivel **10**
4.1. Concepto de float 10
4.2. Inserción de figuras 10
4.3. Inserción de tablas 11
4.4. Inserción de algoritmos 12
4.5. Fuente de figuras y tablas 13
4.6. Ventajas del sistema 13

5. Código y algoritmos **14**
5.1. Entorno lstlisting 14
5.2. Entorno codeblock 14
5.3. Estilo del código 15
5.4. Algoritmos 15
5.5. Buenas prácticas 15

6. Acrónimos y bibliografía **16**
6.1. Acrónimos 16
6.2. Definición de acrónimos 16
6.3. Uso de acrónimos 16
6.4. Bibliografía 17
6.5. Definición de referencias 17
6.6. Citas en el texto 17
6.7. Generación de la bibliografía 18
6.8. Ventajas del sistema 18

7. Entornos de documentación **19**
7.1. Concepto de bloque de documentación 19
7.2. DefinitionBox 19
7.3. NoteBox 19
7.4. WarningBox 20
7.5. ExampleBox 20
7.6. Ventajas 20

8. Modo build **21**
8.1. Concepto de build 21
8.2. Modos de compilación 21
8.3. Modo draft 21
8.4. Modo final 22
8.5. Información de compilación 22
8.6. Metadatos del documento 22
8.7. Log de compilación 23
8.8. Ventajas del sistema 23

9. Guía de uso de la plantilla **24**
9.1. Paso 1 — Obtener la plantilla 24
9.2. Paso 2 — Configurar los metadatos 24
9.3. Paso 3 — Crear capítulos 25
9.4. Paso 4 — Añadir figuras 25
9.5. Paso 5 — Añadir tablas 25
9.6. Paso 6 — Añadir código 26
9.7. Paso 7 — Añadir acrónimos 26
9.8. Paso 8 — Añadir referencias 26
9.9. Paso 9 — Compilar el documento 27
9.10. Resultado 27

10. Buenas prácticas **28**
10.1. Separación de responsabilidades 28
10.2. Archivos que el usuario debe modificar 28
10.3. Archivos que no deberían modificarse 29
10.4. Organización de capítulos 29
10.5. Convenciones de nombres 29
10.6. Errores comunes 30
10.7. Mantenimiento de la plantilla 30

Índice de figuras

4.1. Ejemplo de figura dentro del documento 11

Índice de cuadros

4.1. Ejemplo de tabla 12

Lista de códigos

1.1. Estructura general del proyecto 2
1.2. Bloque de documentación estándar 2
2.1. Estructura de directorios del proyecto 4
2.2. Archivo principal del documento 5
2.3. Flujo de compilación del documento 5
3.1. Creación de un capítulo 7
3.2. Creación de una sección 7
3.3. Creación de una subsección 7
3.4. Estructura típica de un capítulo 8
3.5. Carga de capítulos en main.tex 8
4.1. Macro de inserción de figuras 10
4.2. Ejemplo de inserción de figura 11
4.3. Macro de inserción de tablas 11
4.4. Ejemplo de código tabla 12
4.5. Macro de algoritmos 12
4.6. Ejemplo de algoritmo 12
4.7. Macro de fuente 13
5.1. Ejemplo básico de código 14
5.2. Uso del entorno codeblock 14
5.3. Ejemplo de algoritmo 15
6.1. Definición de acrónimo 16
6.2. Uso de acrónimos 16
6.3. Ejemplo de referencia BibTeX 17
6.4. Cita en el texto 17
6.5. Generación de bibliografía 18
7.1. Uso de definitionbox 19
7.2. Uso de notebox 19
7.3. Uso de warningbox 20
7.4. Uso de examplebox 20
8.1. Activar modo draft 21
8.2. Modo final 22
8.3. Macro de información de build 22
8.4. Metadatos del PDF 22
8.5. Mensaje de compilación 23
9.1. Clonar repositorio 24
9.2. Configuración de metadatos 24
9.3. Crear un capítulo 25
9.4. Cargar capítulo 25
9.5. Insertar figura 25
9.6. Insertar tabla 25
9.7. Bloque de código 26
9.8. Definición de acrónimo 26
9.9. Uso de acrónimo 26

9.10. Referencia BibTeX 26
9.11. Citar referencia 27
9.12. Proceso de compilación 27
10.1. Arquitectura de carpetas 28
10.2. Organización de capítulos 29
10.3. Convención de nombres 29

Lista de algoritmos

1. Busqueda lineal 12

Lista de Acrónimos

API Application Programming Interface

1 Introducción

Esta plantilla ha sido diseñada para facilitar la redacción de documentos académicos y técnicos utilizando LaTeX.

El objetivo principal es proporcionar una estructura modular que permita separar claramente:

- configuración del documento
- estilos tipográficos
- contenido académico
- recursos gráficos y bibliográficos

Gracias a esta separación, el autor puede concentrarse exclusivamente en escribir el contenido del documento sin preocuparse por el formato.

1.1 Objetivo de la plantilla

Definición: Plantilla académica

Una plantilla académica es una estructura predefinida que automatiza la configuración tipográfica y organizativa de un documento.

Esta plantilla está diseñada para:

- trabajos universitarios
- informes técnicos
- proyectos de ingeniería
- Trabajos Fin de Grado (TFG)
- Trabajos Fin de Máster (TFM)

1.2 Filosofía de diseño

La plantilla sigue varios principios de diseño inspirados en prácticas de ingeniería de software.

- modularidad
- separación de responsabilidades
- reutilización
- documentación interna

Nota

El objetivo no es únicamente generar un documento correctamente formateado, sino también demostrar buenas prácticas en la organización de proyectos LaTeX.

1.3 Organización del proyecto

El proyecto se organiza en varias carpetas que separan la configuración del documento del contenido académico.

```
main.tex

latex-university-template/
|-- main.tex
|-- 00_config/
|   |-- config.tex
|   `-- packages.tex
|-- 01_frontmatter/
|-- 02_chapters/
|-- 03_figures/
|-- 04_bibliography/
|-- 05_appendices/
|-- 06_styles/
`-- 07_acronyms/
```

Código 1.1. Estructura general del proyecto

Fuente: Plantilla

1.4 Uso recomendado

El flujo de trabajo recomendado es el siguiente:

1. Configurar los metadatos del documento
2. Escribir el contenido en la carpeta `02_chapters`
3. Añadir figuras en `03_figures`
4. Añadir referencias en `05_bibliography`

Advertencia

Modificar directamente los archivos de configuración o de estilos sin comprender su función puede producir errores de compilación o inconsistencias tipográficas.

1.5 Metodología de documentación

Todos los archivos de la plantilla siguen una convención de comentarios estructurados.

Cada archivo comienza con un bloque de documentación que describe su función dentro del proyecto.

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% FILE: filename.tex
%
% PURPOSE
% Describe el proposito del archivo
```

```
%
% ARCHITECTURE ROLE
% Describe su funcion dentro del sistema
%
% DEPENDENCIES
% Archivos o paquetes necesarios
%
% NOTES FOR DEVELOPERS
% Indicaciones para mantenimiento
%%%
```

2 Arquitectura del proyecto

La plantilla está diseñada siguiendo principios de arquitectura modular inspirados en prácticas de ingeniería de software.

Cada componente del proyecto tiene una responsabilidad claramente definida dentro del sistema.

2.1 Principios de arquitectura

Definición: Arquitectura modular

Una arquitectura modular divide un sistema en componentes independientes que pueden mantenerse o modificarse de forma aislada.

En esta plantilla, el proyecto se divide en varios módulos que gestionan aspectos distintos del documento.

- configuración global
- estilos tipográficos
- contenido académico
- recursos gráficos
- bibliografía
- acrónimos

Nota

Separar estas responsabilidades permite modificar el formato del documento sin alterar el contenido.

2.2 Estructura del proyecto

La estructura de carpetas del proyecto es la siguiente.

```
latex-university-template/
|-- main.tex
|-- 00_config/
|   |-- config.tex
|   `-- packages.tex
|-- 01_frontmatter/
|-- 02_chapters/
|-- 03_figures/
|-- 04_bibliography/
|-- 05_appendices/
|-- 06_styles/
```

```
`-- 07_acronyms/
```

Código 2.1. Estructura de directorios del proyecto

Fuente: Plantilla

Cada carpeta tiene una función específica dentro del sistema.

2.3 Archivo principal

El archivo principal del proyecto es:

```
main.tex
```

Código 2.2. Archivo principal del documento

Fuente: Plantilla

Este archivo controla la estructura general del documento e incluye los distintos capítulos del trabajo.

2.4 Flujo de compilación

Durante la compilación del documento, LaTeX sigue una secuencia similar a la siguiente.

```
main.tex

-> carga configuracion global
-> carga paquetes
-> carga estilos tipograficos
-> carga acronimos
-> genera frontmatter
-> compila capitulos
-> genera bibliografia
-> compila apendices
```

Código 2.3. Flujo de compilación del documento

Fuente: Plantilla

2.5 Responsabilidad de los módulos

- **00_config** Configuración global del documento.
- **01_frontmatter** Portada, dedicatoria, resúmenes.
- **02_chapters** Contenido principal.
- **03_figures** Imágenes del documento.
- **04_bibliography** Base de datos de referencias.
- **05_appendices** Apéndices del documento.
- **06_styles** Definición de estilos tipográficos.
- **07_acronyms** Definición de acrónimos.

2.6 Beneficios de la arquitectura

Esta arquitectura modular ofrece varias ventajas.

- facilita el mantenimiento del proyecto
- permite reutilizar la plantilla en múltiples documentos
- separa claramente contenido y formato
- reduce errores de compilación

Ejemplo

Un autor puede escribir todos los capítulos del documento sin necesidad de modificar los archivos de configuración o los estilos tipográficos.

3 Cómo escribir contenido

El contenido principal del documento se escribe en la carpeta:
`02_chapters`
Cada archivo dentro de esta carpeta representa un capítulo independiente del documento.

3.1 Organización de los capítulos

Un capítulo se define mediante el comando:

```
\chapter{Titulo del capitulo}
```

Código 3.1. Creación de un capítulo

Fuente: Plantilla

Cada capítulo debe almacenarse en un archivo independiente dentro de la carpeta `02_chapters`.

Definición: Capítulo

Un capítulo es la unidad principal de organización del contenido dentro de un documento largo.

3.2 Secciones

Las secciones permiten dividir un capítulo en bloques temáticos.

```
\section{Titulo de la seccion}
```

Código 3.2. Creación de una sección

Fuente: Plantilla

3.3 Subsecciones

Las subsecciones permiten organizar el contenido con mayor nivel de detalle.

```
\subsection{Titulo de la subseccion}
```

Código 3.3. Creación de una subsección

Fuente: Plantilla

Nota

Se recomienda no utilizar más de tres niveles de profundidad en la estructura del documento para mantener la claridad del texto.

3.4 Ejemplo de estructura

Un capítulo típico puede organizarse de la siguiente forma.

```
\chapter{Introduccion}

\section{Contexto}

Texto introductorio.

\section{Objetivos}

\subsection{Objetivo general}

Descripcion.

\subsection{Objetivos especificos}

Descripcion.
```

Código 3.4. Estructura típica de un capítulo

Fuente: Plantilla

3.5 Flujo de escritura recomendado

El flujo de trabajo recomendado al escribir un documento con esta plantilla es el siguiente.

1. Crear un archivo nuevo en la carpeta `02_chapters`
2. Añadir el capítulo en `main.tex`
3. Escribir el contenido utilizando secciones y subsecciones

```
\input{02_chapters/chapter01}
\input{02_chapters/chapter02}
\input{02_chapters/chapter03}
```

Código 3.5. Carga de capítulos en main.tex

Fuente: Plantilla

3.6 Buenas prácticas de escritura

Para mantener la coherencia del documento se recomienda seguir estas prácticas.

- cada capítulo en un archivo independiente
- nombres de archivos descriptivos
- evitar capítulos excesivamente largos
- mantener una jerarquía clara de secciones

Advertencia

Escribir todo el contenido directamente en `main.tex` rompe la arquitectura modular de la plantilla y dificulta el mantenimiento del documento.

3.7 Ejemplo

Ejemplo

Un documento típico puede contener capítulos como:
Introducción
Marco teórico
Metodología
Resultados
Conclusiones

4 Floats de alto nivel

La plantilla proporciona macros que simplifican la inserción de figuras, tablas y algoritmos.

Estas macros encapsulan los entornos estándar de LaTeX y garantizan un formato consistente en todo el documento.

4.1 Concepto de float

Definición: Float

En LaTeX, un *float* es un elemento del documento que puede moverse automáticamente para optimizar el diseño de la página.

Los floats más comunes son:

- figuras
- tablas
- algoritmos

4.2 Inserción de figuras

Las figuras se insertan mediante la macro:

```
\figura{ruta_imagen}{caption}{fuente}{label}
```

Código 4.1. Macro de inserción de figuras

Fuente: Plantilla

Donde:

- **ruta_imagen** es la ruta del archivo de imagen
- **caption** es el título de la figura
- **fuente** indica el origen de la figura
- **label** es la etiqueta para referencias

Figura 4.1. Ejemplo de figura dentro del documento

Fuente: google imagenes

```
\figura{03_figures/borrador.png}{Ejemplo de figura dentro del
    documento}{google imagenes}{fig:demo}
```

Código 4.2. Ejemplo de inserción de figura

Fuente: Plantilla

Nota

Las imágenes utilizadas en el documento deben almacenarse en la carpeta `03_figures`.

4.3 Inserción de tablas

Las tablas se insertan mediante la macro:

```
\tabla{contenido_tabla}{caption}{fuente}{label}
```

Código 4.3. Macro de inserción de tablas

Fuente: Plantilla

Cuadro 4.1. Ejemplo de tabla

Dato 1	Dato 2
Dato 3	Dato 4

Fuente: Plantilla

Ejemplo:

```
\tabla{\centering\begin{tabular}{cc}
\toprule
Dato 1 & Dato 2 \\
\midrule
Dato 3 & Dato 4 \\
\bottomrule
\end{tabular}}{Ejemplo de tabla}{Plantilla}{tablaejemplo}
```

Código 4.4. Ejemplo de código tabla

Fuente: Plantilla

4.4 Inserción de algoritmos

Los algoritmos se insertan mediante la macro:

```
\algoritmo{contenido}{caption}{fuente}{label}
```

Código 4.5. Macro de algoritmos

Fuente: Plantilla

Ejemplo:

```
\algoritmo{
\begin{algorithmic}
\For{i = 1 to n}
\If{A[i] == x}
\Return i
\EndIf
\EndFor
\end{algorithmic}
}{Busqueda lineal}{Plantilla}{busqueda}
```

Código 4.6. Ejemplo de algoritmo

Fuente: Plantilla

Algoritmo 1 Busqueda lineal

for i = 1 to n **do**
  if A[i] == x **then return** i
  end if
end for

Fuente: Plantilla

4.5 Fuente de figuras y tablas

Todas las figuras y tablas incluyen automáticamente una fuente mediante el comando:

```
\source{Plantilla}
```

Código 4.7. Macro de fuente

Fuente: Plantilla

Este comando genera una línea tipográfica centrada debajo del caption que indica el origen del contenido.

4.6 Ventajas del sistema

Utilizar estas macros ofrece varias ventajas.

- formato uniforme en todo el documento
- numeración automática
- inclusión automática de fuentes
- etiquetas consistentes

Advertencia

Insertar figuras o tablas directamente mediante los entornos estándar de LaTeX puede generar inconsistencias de formato dentro del documento.

Ejemplo

Gracias a estas macros, el autor no necesita recordar la estructura completa de los entornos `figure` o `table`, lo que simplifica la escritura del documento.

5 Código y algoritmos

Los documentos técnicos suelen incluir fragmentos de código fuente para explicar algoritmos, configuraciones o ejemplos de implementación.

Esta plantilla proporciona herramientas para mostrar código de forma clara y consistente.

5.1 Entorno lstlisting

El paquete `listings` permite mostrar código fuente dentro del documento.

Definición: Código fuente

Un bloque de código fuente es una representación tipográfica de instrucciones escritas en un lenguaje de programación.

Un bloque de código simple puede escribirse así:

```
for i in range(10):
    print(i)
```

Código 5.1. Ejemplo básico de código

Fuente: Plantilla

5.2 Entorno codeblock

La plantilla define un entorno personalizado llamado `codeblock` que simplifica la inserción de bloques de código.

```
\begin{codeblock}{Ejemplo de codigo}{ejemplo}
for i in range(10):
    print(i)
\end{codeblock}

\source{Plantilla}
```

Código 5.2. Uso del entorno codeblock

Fuente: Plantilla

Este entorno encapsula internamente el entorno `lstlisting` y aplica automáticamente el estilo definido en la plantilla.

Nota

Después del entorno `codeblock` se recomienda añadir siempre el comando `\source` para indicar el origen del código.

5.3 Estilo del código

El estilo de los bloques de código está definido en el archivo:
`07_styles/code_style.tex`
Este estilo incluye:

- fondo ligeramente sombreado
- numeración de líneas
- resaltado de palabras clave
- ajuste automático de líneas largas

Ejemplo

Este estilo está diseñado para mejorar la legibilidad del código dentro de documentos académicos y técnicos.

5.4 Algoritmos

Para describir algoritmos de forma estructurada se utiliza el paquete `algorithm` junto con `algpseudocode`.

Los algoritmos pueden insertarse mediante la macro definida en la plantilla.

```
\algoritmo{
\begin{algorithmic}
\For{i = 1 to n}
\If{A[i] == x}
\Return i
\EndIf
\EndFor
\end{algorithmic}
}{Busqueda lineal}{Plantilla}{busqueda}
```

Código 5.3. Ejemplo de algoritmo

Fuente: Plantilla

5.5 Buenas prácticas

Para mantener la claridad del documento se recomienda:

- incluir solo el código relevante
- evitar bloques de código excesivamente largos
- comentar los fragmentos de código cuando sea necesario

Advertencia

Insertar código sin formato o utilizando entornos verbatim simples reduce la legibilidad del documento.

6 Acrónimos y bibliografía

Los documentos académicos utilizan con frecuencia acrónimos y referencias bibliográficas.

Esta plantilla proporciona un sistema automatizado para gestionar ambos elementos.

6.1 Acrónimos

Definición: Acrónimo

Un acrónimo es una abreviatura formada por las iniciales de una expresión compuesta.
Por ejemplo: API (Application Programming Interface).

Los acrónimos del documento se definen en el archivo:
`07_acronyms/acronyms.tex`

6.2 Definición de acrónimos

Un acrónimo se define mediante el comando:

```
\DeclareAcronym{api}{
  short = API ,
  long  = Application Programming Interface
}
```

Código 6.1. Definición de acrónimo

Fuente: Plantilla

6.3 Uso de acrónimos

Una vez definido el acrónimo, puede utilizarse en el documento mediante el comando:

```
\ac{api}
```

Código 6.2. Uso de acrónimos

Fuente: Plantilla

La primera vez que aparece el acrónimo en el documento, se mostrará de la siguiente forma:

Ejemplo

Application Programming Interface (API)

En las siguientes apariciones, el sistema utilizará únicamente la versión corta.

Por ejemplo:

Ejemplo

Las API permiten que diferentes aplicaciones se comuniquen entre sí.

Nota

La lista completa de acrónimos se genera automáticamente mediante el comando: `\printacronyms`

6.4 Bibliografía

Las referencias bibliográficas se gestionan mediante el paquete `biblatex`.

Definición: Referencia bibliográfica

Una referencia bibliográfica identifica una fuente utilizada en un documento académico, como un libro, artículo científico o página web.

Las referencias se almacenan en el archivo:
`04_bibliography/references.bib`

6.5 Definición de referencias

Una referencia bibliográfica puede definirse de la siguiente forma:

```
@book{knuth1984,
  author = {Donald E. Knuth},
  title = {The TeXbook},
  year = {1984},
  publisher = {Addison-Wesley}
}
```

Código 6.3. Ejemplo de referencia BibTeX

Fuente: Plantilla

6.6 Citas en el texto

Una vez definida la referencia, puede citarse en el documento mediante el comando:

```
\cite{knuth1984}
```

Código 6.4. Cita en el texto

Fuente: Plantilla

Ejemplo dentro del texto:

Ejemplo

El sistema tipográfico TEX fue desarrollado por Donald Knuth Knuth, 1984.

6.7 Generación de la bibliografía

La bibliografía completa del documento se genera automáticamente mediante el comando:

```
\printbibliography
```

Código 6.5. Generación de bibliografía

Fuente: Plantilla

Este comando se encuentra normalmente al final del documento.

Advertencia

Si no se utiliza el compilador `biber`, las referencias bibliográficas no se generarán correctamente.

6.8 Ventajas del sistema

El uso de `biblatex` y `acro` ofrece varias ventajas:

- generación automática de listas
- formato consistente
- reutilización de referencias
- mantenimiento sencillo de grandes documentos

Ejemplo

Este sistema permite gestionar decenas o incluso cientos de referencias bibliográficas sin perder consistencia en el documento.

7 Entornos de documentación

La plantilla incluye entornos visuales diseñados para resaltar información importante dentro del documento.

Estos entornos están inspirados en sistemas modernos de documentación técnica.

7.1 Concepto de bloque de documentación

Definición: Bloque de documentación

Un bloque de documentación es un elemento visual destinado a resaltar información importante dentro de un texto técnico.

Este tipo de bloques mejora la claridad del documento y facilita la lectura.

7.2 DefinitionBox

El entorno `definitionbox` se utiliza para definir conceptos importantes.

```
\begin{definitionbox}[Concepto]
Contenido de la definicion
\end{definitionbox}
```

Código 7.1. Uso de definitionbox

Fuente: Plantilla

Ejemplo real:

Definición: Compilación

La compilación es el proceso mediante el cual LaTeX transforma el código fuente del documento en un archivo PDF final.

7.3 NoteBox

El entorno `notebox` se utiliza para añadir aclaraciones o información adicional.

```
\begin{notebox}
Contenido de la nota
\end{notebox}
```

Código 7.2. Uso de notebox

Fuente: Plantilla

Ejemplo:

Nota

Los bloques de documentación ayudan a mejorar la legibilidad del documento y a destacar información relevante.

7.4 WarningBox

El entorno `warningbox` se utiliza para advertencias importantes.

```
\begin{warningbox}
Mensaje de advertencia
\end{warningbox}
```

Código 7.3. Uso de warningbox

Fuente: Plantilla

Ejemplo:

Advertencia

Modificar los archivos dentro de la carpeta `00_config` sin comprender su función puede provocar errores de compilación.

7.5 ExampleBox

El entorno `examplebox` se utiliza para mostrar ejemplos prácticos.

```
\begin{examplebox}
Contenido del ejemplo
\end{examplebox}
```

Código 7.4. Uso de examplebox

Fuente: Plantilla

Ejemplo:

Ejemplo

Un documento técnico puede utilizar bloques de definición para explicar conceptos clave antes de introducir ejemplos prácticos.

7.6 Ventajas

El uso de estos entornos proporciona varias ventajas.

- mejora la claridad del documento
- permite destacar información importante
- facilita la lectura de documentos largos
- aporta una estética moderna

Nota

Estos bloques son especialmente útiles en documentos docentes, manuales técnicos y material de formación.

8 Modo build

La plantilla incorpora un sistema de compilación que permite generar diferentes versiones del documento.

Este sistema facilita el desarrollo del documento y la generación de la versión final.

8.1 Concepto de build

Definición: Build

Un *build* es el proceso mediante el cual el código fuente de un proyecto se transforma en su resultado final.

En el caso de LaTeX, el build corresponde al proceso de compilación que genera el archivo PDF.

8.2 Modos de compilación

La plantilla define dos modos principales:

- modo **draft**
- modo **final**

8.3 Modo draft

El modo `draft` está diseñado para el proceso de escritura y revisión del documento.

```
\drafttrue
```

Código 8.1. Activar modo draft

Fuente: Plantilla

En este modo el documento puede mostrar información adicional útil durante el desarrollo.

Ejemplo

Ejemplos de información útil en modo draft:

- información de compilación
- fecha de generación del documento
- identificadores internos del documento

8.4 Modo final

El modo final está pensado para la versión definitiva del documento.

```
\draftfalse
```

Código 8.2. Modo final

Fuente: Plantilla

En este modo el documento se genera sin información de depuración ni elementos auxiliares.

Nota

Por defecto la plantilla compila en modo final.

8.5 Información de compilación

La plantilla puede mostrar información sobre el proceso de compilación.

```
\BuildInfo
```

Código 8.3. Macro de información de build

Fuente: Plantilla

Esta información puede incluir:

- fecha de compilación
- versión del documento
- estado del build

8.6 Metadatos del documento

El archivo PDF generado incluye metadatos que describen el documento.
Estos metadatos se configuran mediante:

```
\hypersetup{
  pdftitle={\documenttitle},
  pdfauthor={\documentauthor},
  pdfcreator={mammesa}
}
```

Código 8.4. Metadatos del PDF

Fuente: Plantilla

Definición: Metadatos

Los metadatos son información interna del archivo PDF que describe el contenido del documento.

8.7 Log de compilación

Durante el proceso de compilación, la plantilla puede generar mensajes informativos en el log.

```
\typeout{mammesa-template-loaded}
```

Código 8.5. Mensaje de compilación

Fuente: Plantilla

Este tipo de mensajes permite verificar que los componentes principales de la plantilla se han cargado correctamente.

Advertencia

Los mensajes del log no aparecen en el documento final, pero pueden ser útiles para depurar problemas de compilación.

8.8 Ventajas del sistema

El sistema de build ofrece varias ventajas:

- separación entre documento en desarrollo y final
- control centralizado de la compilación
- información adicional para depuración
- metadatos estructurados del documento

Ejemplo

Este enfoque está inspirado en sistemas de construcción utilizados en proyectos de software, adaptados al flujo de trabajo de LaTeX.

9 Guía de uso de la plantilla

Este capítulo explica paso a paso cómo utilizar la plantilla para escribir un documento académico.

9.1 Paso 1 — Obtener la plantilla

El primer paso consiste en descargar la plantilla.
Esto puede hacerse de dos formas.

- descargando el archivo `.zip` del repositorio
- clonando el repositorio con Git

```
git clone https://github.com/usuario/latex-template.git
```

Código 9.1. Clonar repositorio

Fuente: Plantilla

Una vez descargada la plantilla, el usuario debe abrir el archivo principal:
`main.tex`

9.2 Paso 2 — Configurar los metadatos

Antes de escribir el contenido del documento es necesario configurar los metadatos.
Estos se encuentran en el archivo:
`00_config/config.tex`
Ejemplo:

```
\newcommand{\documenttitle}{Titulo del Trabajo}
\newcommand{\documentauthor}{Nombre del Autor}
\newcommand{\degree}{Grado en Ingenieria Informatica}
\newcommand{\faculty}{Facultad de Ciencias y Tecnologia}
\newcommand{\university}{UNIE Universidad}
```

Código 9.2. Configuración de metadatos

Fuente: Plantilla

Nota

Estos metadatos se utilizan automáticamente en varias partes del documento, como la portada y los metadatos del PDF.

9.3 Paso 3 — Crear capítulos

El contenido principal del documento se escribe en la carpeta:
`02_chapters`
Cada capítulo debe almacenarse en un archivo independiente.
Ejemplo:

```
\chapter{Introduccion}

Este capitulo introduce el tema del documento.
```

Código 9.3. Crear un capítulo

Fuente: Plantilla

Después, el capítulo debe añadirse en el archivo `main.tex`.

```
\input{02_chapters/chapter01}
```

Código 9.4. Cargar capítulo

Fuente: Plantilla

9.4 Paso 4 — Añadir figuras

Las imágenes deben almacenarse en la carpeta:
`03_figures`
Las figuras se insertan mediante la macro:

```
\figura{03_figures/imagen}{
Topologia de red
}{Elaboracion propia}{topologia}
```

Código 9.5. Insertar figura

Fuente: Plantilla

9.5 Paso 5 — Añadir tablas

Las tablas se insertan mediante la macro:

```
\tabla{
\begin{tabular}{cc}
Dato 1 & Dato 2 \\
Dato 3 & Dato 4
\end{tabular}
}{Ejemplo de tabla}{Plantilla}{tablaejemplo}
```

Código 9.6. Insertar tabla

Fuente: Plantilla

9.6 Paso 6 — Añadir código

El código fuente se muestra mediante el entorno `codeblock`.

```
\begin{codeblock}{Ejemplo de codigo}{ejemplo}
for i in range(10):
    print(i)
\end{codeblock}

\source{Plantilla}
```

Código 9.7. Bloque de código

Fuente: Plantilla

9.7 Paso 7 — Añadir acrónimos

Los acrónimos se definen en:
`07_acronyms/acronyms.tex`
Ejemplo:

```
\DeclareAcronym{api}{
  short = API ,
  long  = Application Programming Interface
}
```

Código 9.8. Definición de acrónimo

Fuente: Plantilla

Uso en el documento:

```
\ac{api}
```

Código 9.9. Uso de acrónimo

Fuente: Plantilla

Ejemplo en texto:

Ejemplo

Las API permiten la comunicación entre diferentes sistemas de software.

9.8 Paso 8 — Añadir referencias

Las referencias bibliográficas se almacenan en:
`04_bibliography/references.bib`
Ejemplo de referencia:

```
@book{knuth1984,
  author = {Donald E. Knuth},
  title = {The TeXbook},
  year = {1984},
  publisher = {Addison-Wesley}
}
```

Código 9.10. Referencia BibTeX

Fuente: Plantilla

Uso en el texto:

```
\cite{knuth1984}
```

Código 9.11. Citar referencia

Fuente: Plantilla

Ejemplo:

Ejemplo

El sistema tipográfico TEX fue desarrollado por Donald Knuth Knuth, 1984.

9.9 Paso 9 — Compilar el documento

Para generar el documento final es necesario compilar el proyecto.
El proceso típico es el siguiente:

```
pdflatex main.tex
biber main
pdflatex main.tex
pdflatex main.tex
```

Código 9.12. Proceso de compilación

Fuente: Plantilla

Advertencia

Si no se ejecuta `biber`, las referencias bibliográficas no se generarán correctamente.

9.10 Resultado

Después de completar estos pasos, el documento final se generará como un archivo PDF completamente formateado.

Ejemplo

La estructura modular de la plantilla permite concentrarse en el contenido del documento sin preocuparse por los detalles tipográficos.

10 Buenas prácticas

Este capítulo recoge recomendaciones para utilizar la plantilla de forma eficiente y mantener la consistencia del documento.

10.1 Separación de responsabilidades

Definición: Separación de responsabilidades

La separación de responsabilidades consiste en dividir un sistema en componentes independientes, cada uno encargado de una función específica.

La plantilla está organizada siguiendo este principio.
Cada carpeta tiene un propósito concreto.

```
00_config/
01_frontmatter/
02_chapters/
03_figures/
04_bibliography/
05_appendices/
06_styles/
07_acronyms/
```

Código 10.1. Arquitectura de carpetas

Fuente: Plantilla

Nota

Mantener esta separación facilita el mantenimiento y la reutilización de la plantilla.

10.2 Archivos que el usuario debe modificar

Durante la redacción del documento, el usuario debe trabajar principalmente en los siguientes archivos y carpetas.

- `00_config/config.tex`
- `02_chapters/`
- `03_figures/`
- `04_bibliography/`
- `07_acronyms/`

10.3 Archivos que no deberían modificarse

Los archivos de estilo y configuración avanzada no deberían modificarse salvo que se comprenda su funcionamiento.

- `06_styles/`
- `packages.tex`
- `main.tex`

Advertencia

Modificar archivos de estilo sin comprender su función puede provocar errores de compilación o inconsistencias tipográficas.

10.4 Organización de capítulos

Cada capítulo debe almacenarse en un archivo independiente dentro de la carpeta: `02_chapters`
Ejemplo:

```
chapter01_introduccion.tex
chapter02_arquitectura.tex
chapter03_metodologia.tex
chapter04_resultados.tex
```

Código 10.2. Organización de capítulos

Fuente: Plantilla

Ejemplo

Esta organización permite trabajar en documentos grandes de forma modular y facilita la colaboración entre varios autores.

10.5 Convenciones de nombres

Se recomienda utilizar nombres de archivo descriptivos y consistentes.
Ejemplo:

```
chapter01_introduccion.tex
chapter02_estado_del_arte.tex
chapter03_metodologia.tex
```

Código 10.3. Convención de nombres

Fuente: Plantilla

Esto facilita identificar rápidamente el contenido de cada archivo.

10.6 Errores comunes

Durante el uso de la plantilla pueden aparecer algunos errores frecuentes.

Advertencia

Olvidar ejecutar `biber` provocará que las referencias bibliográficas no aparezcan en el documento final.

Advertencia

Insertar figuras o tablas sin utilizar las macros definidas por la plantilla puede generar inconsistencias de formato.

Advertencia

Editar directamente `main.tex` para escribir contenido rompe la arquitectura modular del proyecto.

10.7 Mantenimiento de la plantilla

Para mantener la plantilla organizada se recomienda:

- documentar los cambios realizados
- mantener comentarios claros en los archivos
- respetar la estructura de carpetas
- evitar modificaciones innecesarias en los estilos

Ejemplo

Una plantilla bien mantenida puede reutilizarse en múltiples proyectos académicos y técnicos.

Bibliografía

Knuth, D. (1984). *The TeXbook*. Addison-Wesley.